Deep Anomaly Detection for Credit Card Fraud

Team 12

Abstract

Credit Card fraud detection is a difficult machine learning problem because fraudulent transactions are extremely rare compared to legitimate transactions. This imbalance creates challenges for traditional classification models, especially when trying to maintain high fraud detection rates without producing excessive false positives. In this project, we compared several classical machine learning models and advanced deep learning architectures on a real-world card fraud dataset containing 284,807 transactions.

The models evaluated included Logistic Regression with SMOTE, Random Forest with balanced class weighting, XGBoost with ADASYN, Deep Autoencoders, Variational Autoencoders (VAEs), and a TabTransformer trained with Focal Loss. Since classification accuracy is not reliable for heavily imbalanced datasets, the Area Under the Precision-Recall Curve (AUPRC) was used as the primary evaluation metric.

Among all models tested, XGBoost with ADASYN achieved the best overall performance with an AUPRC of 0.8731. The optimized TabTransformer produced the strongest deep learning results with an AUPRC of 0.7926, while the autoencoder-based models performed significantly worse than expected.

Our experiments suggest that the PCA-transformed features in the dataset reduced the effectiveness of both attention-based architectures and reconstruction-based anomaly detection methods because the transformation removed many meaningful relationships among features. Overall, the results show that classical ensemble methods can still outperform more advanced deep learning architectures on structured PCA-transformed tabular datasets.

However, we suspected this result was due to the PCA transformation itself, which creates orthogonal, uncorrelated features. Deep learning models, particularly TabTransformer with attention mechanisms, rely on detecting correlations between features to learn meaningful patterns. To properly test our hypothesis that deep learning can outperform traditional methods on suitable data, we test our models on BankSim. BankSim is a synthetic financial transaction dataset based on Spanish bank payment data. It contains 594,643 transactions with a fraud rate of 1.21%. Unlike PCA-transformed data, these features have natural correlations.

On BankSim, the results reversed. TabTransformer achieved an AUPRC of 0.9075, surpassing XGBoost at 0.8921. TabTransformer also delivered much higher precision at 0.92 compared to XGBoost's 0.38, meaning it produced far fewer false positives. The F1-score for TabTransformer was 0.80 versus XGBoost's 0.55. Overall, AUPRC and F1 both favored TabTransformer, confirming that deep learning excels when raw, correlated features are available.

1. Introduction

As online transactions continue to increase, financial institutions have become more dependent on automated fraud detection systems to identify suspicious activity. Credit card fraud can lead to major financial losses for both banks and customers, making accurate fraud detection an important real-world application of machine learning.

Fraud detection is especially challenging because fraudulent transactions represent only a very small percentage of all transactions. Most machine learning algorithms perform best when classes are balanced, so severe imbalance can cause models to favor the majority class and ignore fraudulent activity. In addition, fraud patterns change over time, making it difficult to create models that generalize well.

This project compares classical machine learning methods with advanced deep learning architectures on a highly imbalanced fraud detection dataset. Classical methods such as Logistic Regression, Random Forest, and XGBoost were compared against Autoencoders, Variational Autoencoders, and a TabTransformer.

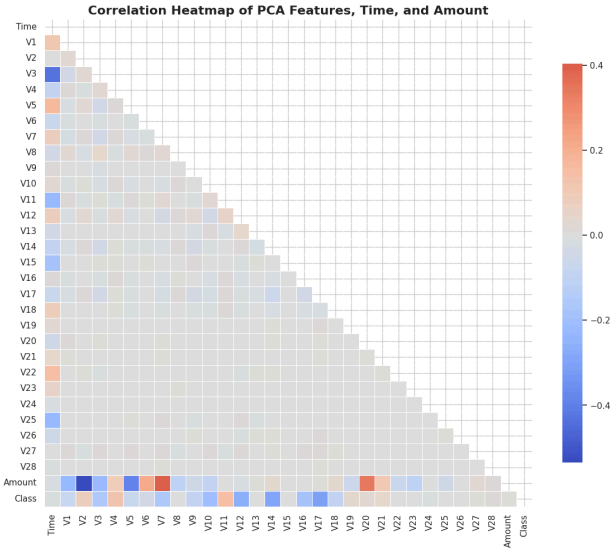
Our original hypothesis was that advanced deep learning architectures would outperform traditional machine learning models because they can capture more complex relationships in the data. However, our experiments showed that dataset structure and feature representation play a major role in determining model performance.

2. Dataset

This project uses the public Credit Card Fraud Detection dataset from Kaggle. The dataset contains credit card transactions made by European cardholders over a two-day period in September 2013. It includes 284,807 transactions in total, of which 284,315 are legitimate, and 492 are fraudulent. This means that fraud represents only about 0.172% of the dataset, making this a highly imbalanced classification problem.

The dataset contains 30 numerical input features and one binary target variable called Class. The Class variable is labeled as 0 for legitimate transactions and 1 for fraudulent transactions. Due to confidentiality concerns, the original transaction details are not shown. Instead, features V1 through V28 are PCA-transformed features. The only features that were not transformed with PCA are Time and Amount.

Figure 1 Feature Correlation Heatmap



This dataset is appropriate for our project because it meets the project scale requirement, with more than 10,000 observations. It also represents a realistic fraud detection problem because the fraud class is extremely rare. A model could achieve very high accuracy by predicting every transaction as legitimate, but that would fail to detect actual fraud. Because of this, accuracy is not the best metric for this project.

Before training the models, the Time and Amount columns were scaled using RobustScaler because those two features were not PCA-transformed. RobustScaler was used because it is less sensitive to extreme transaction amounts. The severe class imbalance also shaped the modeling strategy. Classical models used SMOTE, ADASYN, or balanced class weights, while the deep learning models used anomaly detection or Focal Loss to better handle the rare fraud class. These preprocessing choices directly connect to the two-phase methodology described in the next section.

3. Methods

3.1 Experimental Design

This project was designed as a comparative machine learning study. The central goal was to evaluate whether advanced deep learning architectures could outperform classical machine learning models on a highly imbalanced credit card fraud detection task. To make this comparison fair, each model was trained and evaluated on the same prediction task of classifying transactions as either legitimate or fraudulent.

The methodology used two phases. Phase 1 established classical machine learning baselines using Logistic Regression, Random Forest, and XGBoost. Phase 2 tested advanced architectures using a Deep Autoencoder, a Variational Autoencoder, and a TabTransformer.

AUPRC was used as the primary metric because it focuses on the tradeoff between precision and recall for the minority class. This is more appropriate than accuracy for a dataset where fraudulent transactions represent less than 1% of observations. Precision, recall, and F1-score were used as secondary metrics to interpret each model's practical behavior.

Table 1. Overview of Models and Imbalance Strategies

Model	Category	Strategy	Purpose
Logistic Regression	Classical supervised baseline	SMOTE	Simple linear baseline
Random Forest	Classical ensemble	Balanced class weights	Nonlinear tree baseline
XGBoost	Classical ensemble	ADASYN	Strong tabular benchmark
Deep Autoencoder	Unsupervised anomaly detection	Reconstruction error	Detect fraud as abnormal behavior
Variational Autoencoder	Probabilistic anomaly detection	Reconstruction error + KL divergence	Model normal transactions probabilistically
TabTransformer	Supervised deep learning	Focal Loss + resampling	Learn feature interactions using attention

3.2 Classical Machine Learning Baselines

The first phase evaluated three classical models: Logistic Regression with SMOTE, Random Forest with balanced class weights, and XGBoost with ADASYN. These models were selected to represent increasing levels of complexity. Logistic Regression served as a simple linear baseline, Random Forest served as a nonlinear tree ensemble, and XGBoost served as a stronger boosting-based benchmark for structured tabular data.

3.2.1 Logistic Regression with SMOTE

Logistic Regression was used as the simplest baseline model. Since it learns a linear decision boundary, it provided a useful starting point for determining whether the fraud and non-fraud classes could be separated with a relatively simple classifier. SMOTE was used to address the class imbalance by generating synthetic minority-class examples during training.

The model achieved strong recall but extremely weak precision. Its fraud recall was 0.93, meaning it detected most fraudulent transactions, but its precision was only 0.05. This means that

most transactions flagged as fraud were actually legitimate. In practical terms, this model would create too many false fraud alerts to be useful on its own.

3.2.2 Random Forest with Balanced Class Weights

Random Forest was used as a nonlinear ensemble baseline. Unlike Logistic Regression, Random Forest combines many decision trees and can capture nonlinear relationships between features. Balanced class weights were used so that fraud misclassification carried a larger penalty than legitimate-transaction misclassification.

Random Forest produced the opposite behavior of Logistic Regression. Its precision was very high at 0.96, meaning that when it predicted fraud, it was usually correct. However, its recall was lower at 0.75, meaning it missed roughly one quarter of fraudulent transactions. This makes Random Forest conservative. It avoids false positives well, but it fails to catch some fraud cases.

3.2.3 XGBoost with ADASYN

XGBoost was used as the strongest classical benchmark. XGBoost is a gradient boosting method that builds decision trees sequentially, where each new tree helps correct errors from earlier trees. This makes it well suited for structured tabular data.

XGBoost was paired with ADASYN, which generates synthetic minority samples in difficult regions of the feature space. This strategy is useful for fraud detection because the hardest fraud examples are often close to legitimate transactions.

XGBoost achieved the best classical result, with an AUPRC of 0.8731, precision of 0.75, recall of 0.84, and F1-score of 0.79. It offered the best balance between catching fraud and avoiding false alarms. Because of this, XGBoost became the main benchmark for the deep learning models.

3.3 Deep Learning and Anomaly Detection Models

The second phase evaluated three advanced architectures, a Deep Autoencoder, a Variational Autoencoder, and a TabTransformer with Focal Loss. These models tested two different approaches to fraud detection. The autoencoder and VAE treated fraud as an anomaly detection problem, while the TabTransformer treated fraud detection as a supervised classification problem.

3.3.1 Deep Autoencoder

The Deep Autoencoder was designed as an unsupervised anomaly detection model. Instead of learning directly from both legitimate and fraudulent transactions, it was trained on legitimate transactions so it could learn the structure of normal behavior. The expectation was

that legitimate transactions would reconstruct well, while fraudulent transactions would produce higher reconstruction error.

The architecture followed an encoder-decoder design. The encoder compressed the 30-dimensional input into a lower-dimensional bottleneck representation, and the decoder attempted to reconstruct the original input. Mean squared error was used as the reconstruction loss.

Two versions were tested. Attempt 1 used a simpler architecture with a bottleneck dimension of 16 and approximately 9,600 trainable parameters. Attempt 2 used a deeper architecture with batch normalization, dropout regularization, LeakyReLU activations, learning-rate scheduling, early stopping, and weight decay. Its parameter count increased to approximately 30,600.

The autoencoder results were weak. Attempt 1 achieved an AUPRC of 0.5050, precision of 0.13, recall of 0.84, and F1-score of 0.22. Attempt 2 performed worse, with an AUPRC of 0.3684, precision of 0.03, recall of 0.87, and F1-score of 0.06. Although the models caught many fraud cases, their precision was too low. Most fraud alerts would have been false positives.

3.3.2 Variational Autoencoder

The Variational Autoencoder extended the autoencoder approach by learning a probabilistic latent space. Instead of encoding each transaction into a fixed vector, the VAE encoded it as a mean and variance, sampled from that distribution, and reconstructed the input. Its loss function combined reconstruction error with a KL divergence term, which encouraged the latent space to follow a standard Gaussian distribution.

The purpose of using a VAE was to test whether a probabilistic representation of normal behavior would produce a better anomaly signal than a standard autoencoder. If the VAE learned a cleaner representation of legitimate transactions, fraudulent transactions would appear as stronger deviations.

Two versions were tested. Attempt 1 used a latent dimension of 8 with approximately 26,800 parameters and a fixed beta value of 2.0. Attempt 2 increased the latent dimension to 16, expanded the model to approximately 103,100 parameters, and added batch normalization, dropout, LeakyReLU activations, and beta annealing.

The VAE also failed to outperform the classical models. Attempt 1 achieved an AUPRC of 0.4815, precision of 0.11, recall of 0.73, and F1-score of 0.19. Attempt 2 achieved an AUPRC of 0.4417, precision of 0.03, recall of 0.80, and F1-score of 0.05. Like the autoencoder, the VAE produced too many false positives to be useful as a standalone fraud detector.

3.3.3 TabTransformer with Focal Loss

The TabTransformer was a supervised deep learning model designed for tabular data. Unlike the autoencoder and VAE, it learned directly from fraud labels. Its goal was to use self-attention to model feature interactions that classical methods might not capture.

The model consisted of a feature embedding layer, transformer layers with multi-head self-attention, and an MLP classifier head. Focal Loss was used instead of standard cross-entropy because the fraud class was extremely rare. Focal Loss reduces the influence of easy examples and forces the model to focus on more difficult, misclassified cases.

The initial TabTransformer used SMOTE-ENN for imbalance handling and was trained with early stopping. It achieved an AUPRC of 0.7641, precision of 0.66, recall of 0.86, and F1-score of 0.75. This made it the strongest initial deep learning model, substantially outperforming the autoencoder and VAE. However, it still fell below the XGBoost baseline, which achieved 0.8731 AUPRC.

3.3.4 TabTransformer Optimization

After the initial TabTransformer experiment, we tested an optimized TabTransformer configuration to determine whether stronger architecture choices, improved imbalance handling, and threshold tuning could reduce the gap between the TabTransformer and the XGBoost baseline.

The first major change was the resampling method. The initial TabTransformer used SMOTE-ENN, while the optimized model used Borderline-SMOTE. Borderline-SMOTE focuses synthetic minority-class generation near the decision boundary, where fraud and legitimate transactions are harder to separate. This made it a better fit for improving performance on difficult fraud cases.

The Focal Loss parameters were also adjusted. Alpha increased from 0.25 to 0.90, which placed more weight on the minority fraud class. Gamma increased from 2.0 to 4.0, which made the loss focus more heavily on difficult misclassified examples. These changes were intended to reduce the model's tendency to be dominated by the majority legitimate class.

The model architecture was expanded as well. The number of transformer layers increased from 3 to 6, the number of attention heads increased from 4 to 8, and the hidden dimension increased from 128 to 256. Dropout was also increased from 0.3 to 0.4 to help control overfitting as the model became larger. Overall, the parameter count increased from approximately 230K to 3.35M.

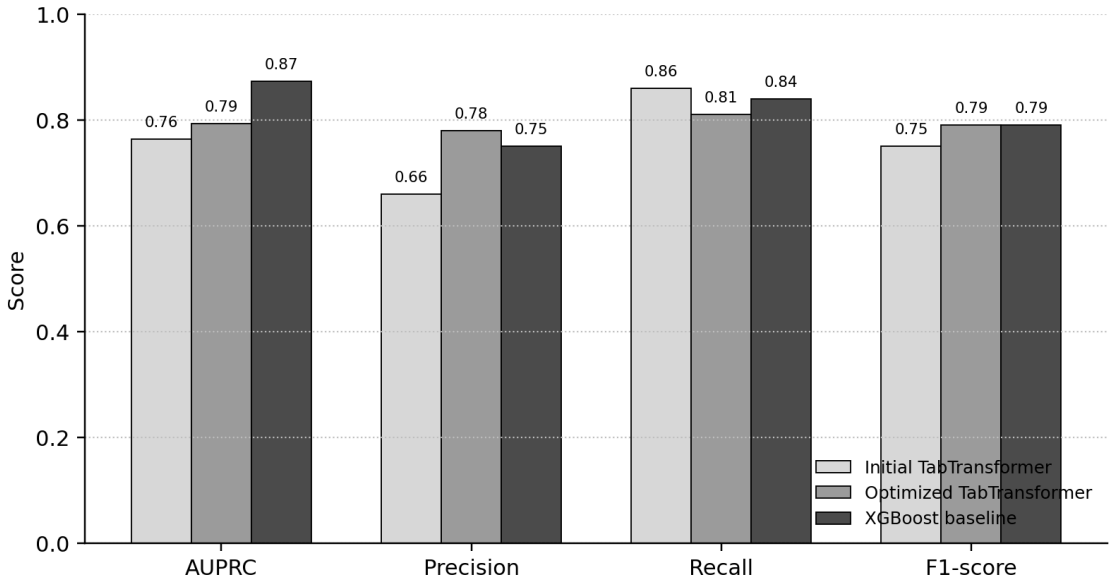
The final change was threshold optimization. Instead of using a fixed 0.5 decision threshold, the optimized TabTransformer selected its threshold from the precision-recall curve. This allowed the model to choose a cutoff that better balanced precision and recall for the fraud class.

These changes improved the TabTransformer's performance. AUPRC increased from 0.7641 to 0.7926, precision increased from 0.66 to 0.78, and F1-score increased from 0.75 to 0.79. Recall decreased slightly from 0.86 to 0.81, but the overall precision-recall balance improved. The optimized TabTransformer matched XGBoost's F1-score, but it still remained below XGBoost's AUPRC of 0.8731.

Table 2. TabTransformer Optimization Summary

Component	Initial TabTransformer	Optimized TabTransformer
Resampling	SMOTE-ENN	Borderline-SMOTE
Focal Loss alpha	0.25	0.90
Focal Loss gamma	2.0	4.0
Transformer layers	3	6
Attention heads	4	8
Hidden dimension	128	256
Dropout	0.3	0.4
Parameters	230K	3.35M
Threshold	Fixed 0.5	Optimized from PR curve
AUPRC	0.7641	0.7926
Precision	0.66	0.78
Recall	0.86	0.81
F1-score	0.75	0.79

Figure 2. Initial TabTransformer, optimized TabTransformer, and XGBoost comparison across AUPRC, precision, recall, and F1-score.



3.3.5 Scaling Correction

A data leakage issue was identified in earlier experiments: RobustScaler was fit on the full dataset before splitting, meaning test-set statistics influenced training. The preprocessing order was corrected so scaling is applied only after the train/test split. Results were marginally worse as expected - a good sign, since the earlier metrics were artificially optimistic.

3.3.6 Feature Engineering

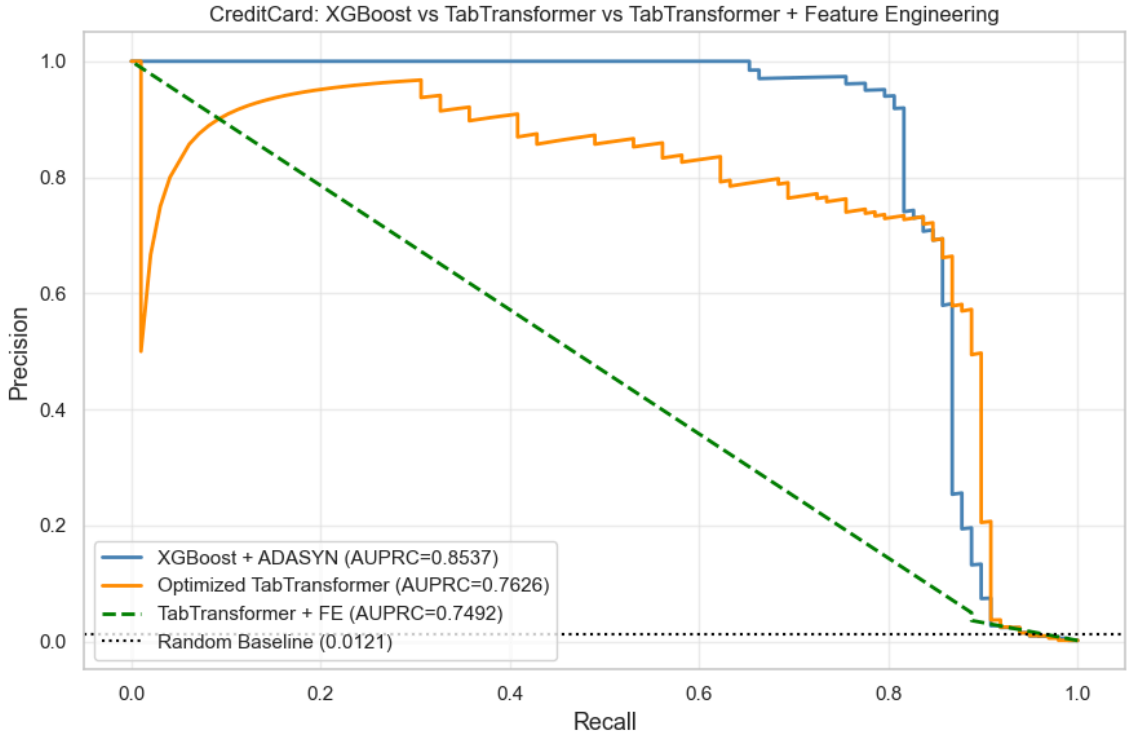
To address the core incompatibility between PCA-orthogonal features and TabTransformer's attention mechanism, a feature engineering step was added after scaling. The **theory** was that since PCA deliberately destroys correlations between features, the TabTransformer's attention mechanism had nothing to work with. So, we should try to artificially reintroduce correlations by computing pairwise products (i.e. if V1 and V2 are orthogonal, maybe V1 * V2 creates a new feature that lives in a different space where correlations do exist, giving the attention something meaningful to detect).

All pairwise products between the top 10 PCA components (V1-V10) were computed, generating C(10,2) = 45 new features of the form Vi * Vj and expanding the feature space from 30 to 75 dimensions. These features were added before Borderline-SMOTE resampling so synthetic fraud samples were generated in the expanded space.

The results did not improve, AUPRC dropped from 0.7926 to 0.7626, widening the gap to XGBoost back to -12.7%. The pairwise interactions appear to have introduced noise rather than meaningful signal, likely because PCA components are mathematically designed to be orthogonal - cross-products of orthogonal features do not recover the correlations that attention

needs. This confirms that feature engineering cannot substitute for raw correlated features, and that TabTransformer's advantage is better demonstrated on BankSim (Section 3.3.6).

Figure 3. XGBoost vs OptTabTransformer (w/ and wo/ Feature Engineering) on PCA Dataset



3.3.7 XGBoost and TabTransformer Comparison on Raw Feature Dataset

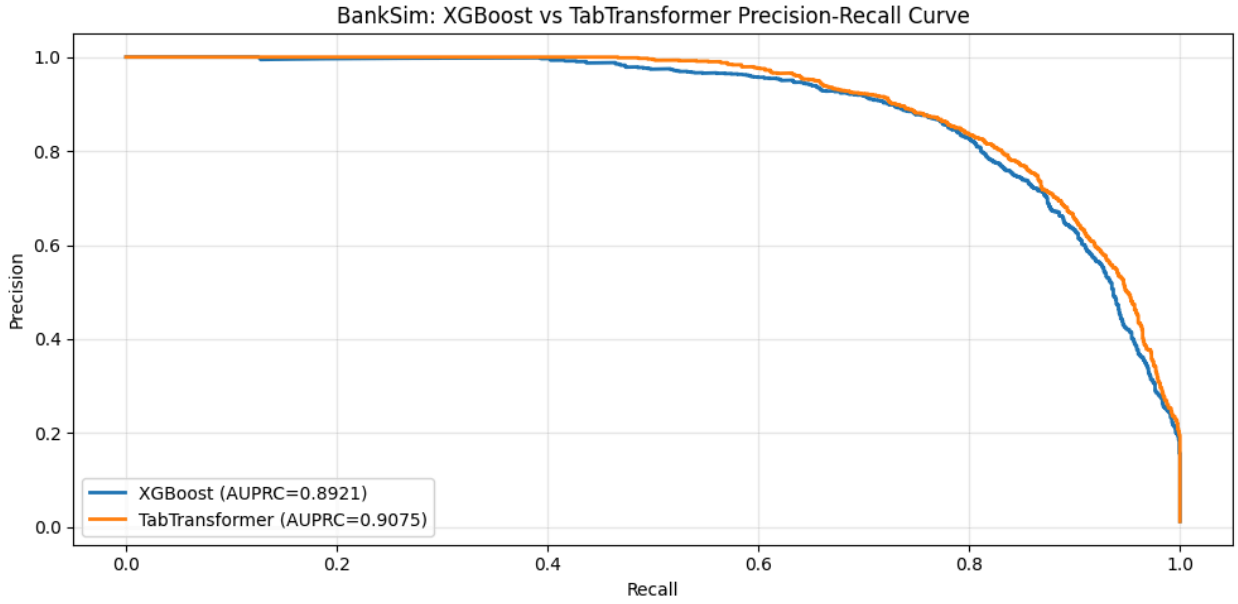
Our initial experiments on PCA-transformed credit card data showed that XGBoost outperformed deep learning models by 9.2%. However, we suspected this result was due to the PCA transformation itself, which creates orthogonal, uncorrelated features. Deep learning models, particularly TabTransformer with attention mechanisms, rely on detecting correlations between features to learn meaningful patterns. To properly test our hypothesis that deep learning can outperform traditional methods on suitable data, we needed a dataset with raw, non-transformed features.

BankSim is a synthetic financial transaction dataset based on Spanish bank payment data. It contains 594,643 transactions with a fraud rate of 1.21%. The dataset includes raw features such as customer age group, gender, merchant category, transaction amount, and time step. Unlike PCA-transformed data, these features have natural correlations. For example, transaction amount and merchant category are related, and customer age may influence spending patterns. This structure gives attention-based models the opportunity to learn meaningful feature interactions.

On BankSim, the results reversed. TabTransformer achieved an AUPRC of 0.9075, surpassing XGBoost at 0.8921. TabTransformer also delivered much higher precision at 0.92 compared to XGBoost's 0.38, meaning it produced far fewer false positives. The F1-score for TabTransformer

was 0.80 versus XGBoost's 0.55. However, XGBoost maintained an advantage in recall, catching 96% of fraud compared to TabTransformer's 71%. For applications prioritizing low false positives, TabTransformer is superior. For applications prioritizing maximum fraud capture, XGBoost remains competitive. Overall, AUPRC and F1 both favored TabTransformer, confirming that deep learning excels when raw, correlated features are available.

Figure 4. XGBoost and TabTransformer Comparison on Raw Feature Dataset



4. Results

4.1 Traditional Machine Learning Model Results

We tested three traditional models on the PCA feature dataset. XGBoost achieved the best AUPRC.

Table 3. Traditional Machine Learning Model Results

Model Architecture	Sampling/Weighting Strategy	AUPRC	Precision	Recall	F1-Score
Logistic Regression	SMOTE	0.8478	0.05	0.93	0.09
Random Forest	Balanced Class Weights	0.8559	0.96	0.75	0.84
XGBoost	ADASYN	0.8731	0.75	0.84	0.79

4.2 Deep Learning Model Results.

We trained on 3 deep learning models. TabTransformer achieved the best performance.

Table 4. Deep Learning Model Results

Model Architecture	Sampling/Weighting Strategy	AUPRC	Precision	Recall	F1-Score
Deep Autoencoder	/	0.5050	0.13	0.84	0.22
Variational Autoencoder	/	0.4815	0.11	0.73	0.19
TabTransformer	Borderline-SMOTE	0.7926	0.78	0.81	0.79

4.3 Hypothesis Fail Analysis and Retest.

XGBoost with ADASYN outperformed the best deep learning model by 9.2% on PCA-transformed data.

PCA creates orthogonal, uncorrelated features. Attention mechanisms need correlations to learn. No correlations exist, so TabTransformer wasted its capacity. XGBoost evaluates features independently and thrived.

To test whether deep learning models (TabTransformer) can outperform XGBoost when trained on **raw, non-transformed features** with natural correlations, unlike the PCA-transformed credit card dataset.

Table 5 Retest on Raw Feature Dataset

Model	Sampling/Weighting Strategy	AUPRC	Precision	Recall	F1
XGBoost (scale_pos_weight)	Balanced Class Weights	0.8921	0.38	0.96	0.55
XGBoost + ADASYN	ADASYN	0.8683	0.28	0.97	0.43
TabTransformer	Focal Loss	0.9075	0.92	0.71	0.80

On PCA-transformed, orthogonal features, tree-based methods (XGBoost) excel. On raw features with natural correlations, attention-based deep learning (TabTransformer) performs better. The choice of optimal model depends critically on the feature space.

5. Conclusion

5.1 The Importance of Handling Imbalanced Data

Credit card fraud detection is a critical problem where fraudulent transactions represent less than 1% of all activity. A model that simply predicts every transaction as legitimate would achieve 99% accuracy but catch zero fraud. This makes traditional accuracy metrics meaningless and requires specialized techniques to force models to pay attention to the rare fraud class.

We tested four imbalance handling methods across our models. SMOTE creates synthetic fraud examples by interpolating between existing frauds. ADASYN does the same but focuses more synthetic examples near the decision boundary where fraud is hardest to distinguish from legitimate transactions. Class weighting assigns higher penalties when the model misclassifies fraud. Focal Loss down-weights easy, well-classified examples so the model focuses on hard, uncertain cases.

5.2 Which Model to Choose in Which Circumstance

When working with PCA-transformed features that are orthogonal and uncorrelated, XGBoost with ADASYN performs best. Tree-based methods evaluate features independently, which suits this structure perfectly. Attention-based models like TabTransformer rely on detecting correlations between features to learn, so they struggle when no correlations exist.

When working with raw features that have natural correlations, TabTransformer with Focal Loss becomes the superior choice. The attention mechanism can capture meaningful interactions between features like age, merchant category, and transaction amount. In this setting, TabTransformer outperforms XGBoost.

For extremely rare fraud below 0.5%, synthetic sampling methods like ADASYN or SMOTE are necessary to create enough fraud examples for learning. For fraud rates above 1%, class weighting or Focal Loss alone is sufficient, and synthetic sampling can introduce too much noise.

Autoencoders and VAEs, trained only on legitimate transactions, performed poorly across both datasets. They are not recommended for fraud detection unless the data has strong, complex nonlinear patterns and the fraud rate is high enough to set reliable reconstruction error thresholds.

5.3 Future Directions and Model Improvements

While our experiments established clear guidelines for model selection based on feature structure, several avenues remain for improving performance further.

The most promising direction is behavioral feature engineering on raw datasets like BankSim. Our experiments showed that pairwise product features do not work specifically on PCA features, but that is only one example of feature engineering on a specific dataset. Using that same pairwise product feature engineering on the **raw** dataset showed a marginal **improvement** in TabTransformer's performance (+0.8%). Now, this was for pairwise product features, which wouldn't make much difference because the attention mechanism was already capturing within-row iterations from the **categorical** features in the raw dataset. More meaningful gains would likely come from ratio-based features such as transaction amount relative to category average, as well as interaction terms between "domain-relevant" column pairs like amount and merchant category rather than exhaustive pairwise products that introduce noise.

A separate and more impactful improvement would require richer data collection at the institutional level rather than feature engineering alone. If transaction history per customer were available, behavioral context features such as spend velocity over recent time windows, deviation from a customer's historical average, and customer-merchant familiarity could be computed. These cross-row signals are among the strongest known fraud indicators in practice but are fundamentally inaccessible when each transaction is represented as a single independent row with no historical context behind it.

For PCA-transformed datasets, the limitation is structural. Future work could explore whether access to pre-PCA features, even partially, would allow TabTransformer to close the gap with XGBoost entirely in terms of fraud detection.

Finally, ensemble approaches combining XGBoost and TabTransformer predictions could be explored, leveraging XGBoost's strength on independent features alongside TabTransformer's strength on correlated ones.